

PRACTICAL ACTIVITY 3: Generating a Hash and HMAC

Objective:

To generate a cryptographic hash and a Hash-based Message Authentication Code (HMAC) using Python, and to understand how these mechanisms are used to ensure data integrity and authentication in secure systems.

Tools & Setup:

LANGUAGE: Python

LIBRARY: hashlib, hmac

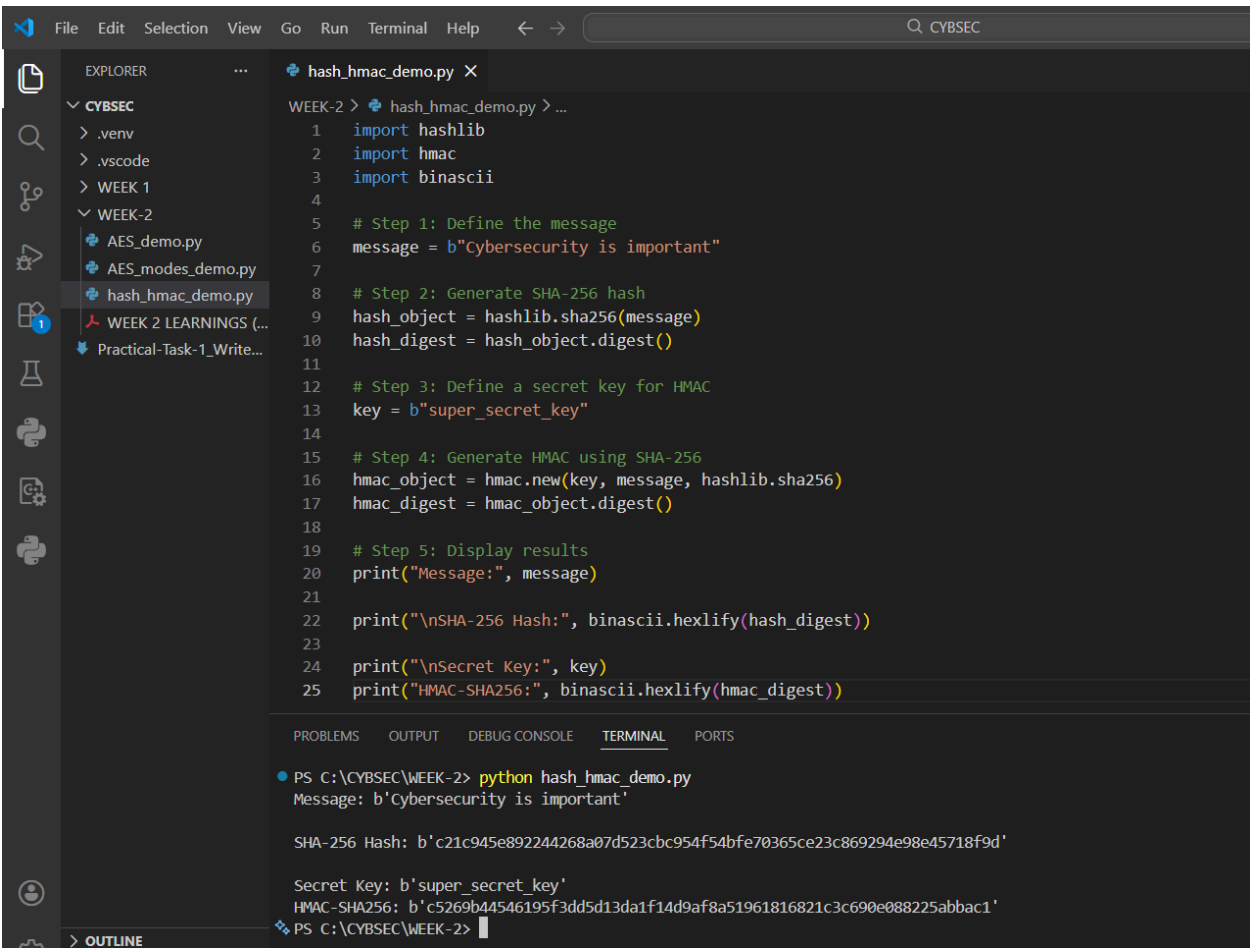
ENVIRONMENT: Local machine using a Python script

PROCEDURE:

To begin the activity, I created a new Python file named **hash_hmac_demo.py**. In this script, I imported the built-in Python modules **hashlib** and **hmac**, which provide functions for computing cryptographic hashes and message authentication codes.

First, I defined a simple message that would serve as the input data:

```
message = b"Cybersecurity is important"
```



The screenshot shows a Visual Studio Code editor with a file named `hash_hmac_demo.py` open. The file explorer on the left shows a project structure with folders `CYBSEC`, `WEEK 1`, and `WEEK-2`. The `hash_hmac_demo.py` file is selected under `WEEK-2`. The code in the editor is as follows:

```
1 import hashlib
2 import hmac
3 import binascii
4
5 # Step 1: Define the message
6 message = b"Cybersecurity is important"
7
8 # Step 2: Generate SHA-256 hash
9 hash_object = hashlib.sha256(message)
10 hash_digest = hash_object.digest()
11
12 # Step 3: Define a secret key for HMAC
13 key = b"super_secret_key"
14
15 # Step 4: Generate HMAC using SHA-256
16 hmac_object = hmac.new(key, message, hashlib.sha256)
17 hmac_digest = hmac_object.digest()
18
19 # Step 5: Display results
20 print("Message:", message)
21
22 print("\nSHA-256 Hash:", binascii.hexlify(hash_digest))
23
24 print("\nSecret Key:", key)
25 print("\nHMAC-SHA256:", binascii.hexlify(hmac_digest))
```

The terminal at the bottom shows the output of running the script:

```
PS C:\CYBSEC\WEEK-2> python hash_hmac_demo.py
Message: b'Cybersecurity is important'

SHA-256 Hash: b'c21c945e892244268a07d523cbc954f54bfe70365ce23c869294e98e45718f9d'

Secret Key: b'super_secret_key'
HMAC-SHA256: b'c5269b44546195f3dd5d13da1f14d9af8a51961816821c3c690e088225abbac1'
PS C:\CYBSEC\WEEK-2>
```

Generating a Hash:

Next, I generated a cryptographic hash of the message using the **SHA-256 hashing algorithm** from the `hashlib` library. A hash function converts an input message into a fixed-length output known as a **digest**. The key characteristic of a cryptographic hash is that it is **one-way**, meaning the original message cannot be recovered from the hash.

The program computed the SHA-256 hash and displayed the result in hexadecimal format. Even a small change in the input message would produce a completely different hash value, demonstrating the sensitivity of hash functions to input changes.

Generating an HMAC:

After generating the hash, I implemented a **Hash-based Message Authentication Code (HMAC)**. Unlike a regular hash, HMAC uses both the message and a **secret key** to generate the authentication code.

To do this, I defined a secret key: `key = b"secret_key"`. Then, using the `hmac` library with the SHA-256 hashing algorithm, I computed the HMAC value of the message. The resulting output was also displayed in hexadecimal form.

The HMAC ensures not only the integrity of the message but also its **authenticity**, since only someone who possesses the secret key can generate the correct HMAC value.

Verification:

Finally, I verified that the HMAC could be reproduced using the same key and message. If either the message or the key were changed, the resulting HMAC would be different. This demonstrates how HMAC can detect tampering and confirm the identity of the sender.

Reflection:

This activity helped me understand the difference between hash functions and HMACs, and how they are used in modern security systems. A cryptographic hash ensures data integrity by allowing systems to detect whether information has been altered. However, a regular hash alone does not provide authentication because anyone can compute the same hash from the same message.

HMAC addresses this limitation by incorporating a secret key into the hashing process. Because the key is known only to trusted parties, the generated HMAC proves both the integrity of the message and the authenticity of the sender.

In real-world applications, hashes are commonly used for tasks such as password storage, file integrity verification, and digital fingerprints, while HMAC is used in secure communications, API authentication, and network protocols such as TLS and HTTPS.

Through this activity, I gained practical experience generating and verifying hashes and HMAC values, which reinforced how cryptographic primitives help protect data integrity and authenticity in modern cybersecurity systems.